22/10/24

Activity on SLR and CLR parsing

LR(0) & LR(1) parsing for the given grammar

Given grammar

$S \rightarrow AA$
$A \rightarrow aA \mid b$

We have to construct the LR(0) items, state transition diagrams, parsing tables, and parse the input string abab using both LR(0) and LR(1)

Step 1: Augment the grammar

Add a new start production $S' \rightarrow S$

Production number	Production
(0)	$S' \rightarrow S$
(1)	$S \rightarrow AA$
(2)	$A \rightarrow aA$
(3)	$A \rightarrow b$

The input string is abab, with $ as the end-of input marker. The augmented input is abab$.

Part 1: LR(0) Parsing

1. Construct the LR(0) items

An LR(0) items is obtained by placing a dot (·) at every possible position in a production.

Canonical collection of LR(0) items

I_0 = closure ({S' → .S})

S' → .S
S → .AA
A → .aA
A → .b

I_1 = goto (I_0, S)

S' → S.

I_2 = goto (I_0, A)

S → A.A
A → .aA
A → .b

I_3 = goto (I_0, a)

A → a.A
A → .aA
A → .b

I_4 = goto (I_0, b)

A → b.

I_5 = goto (I_2, A)

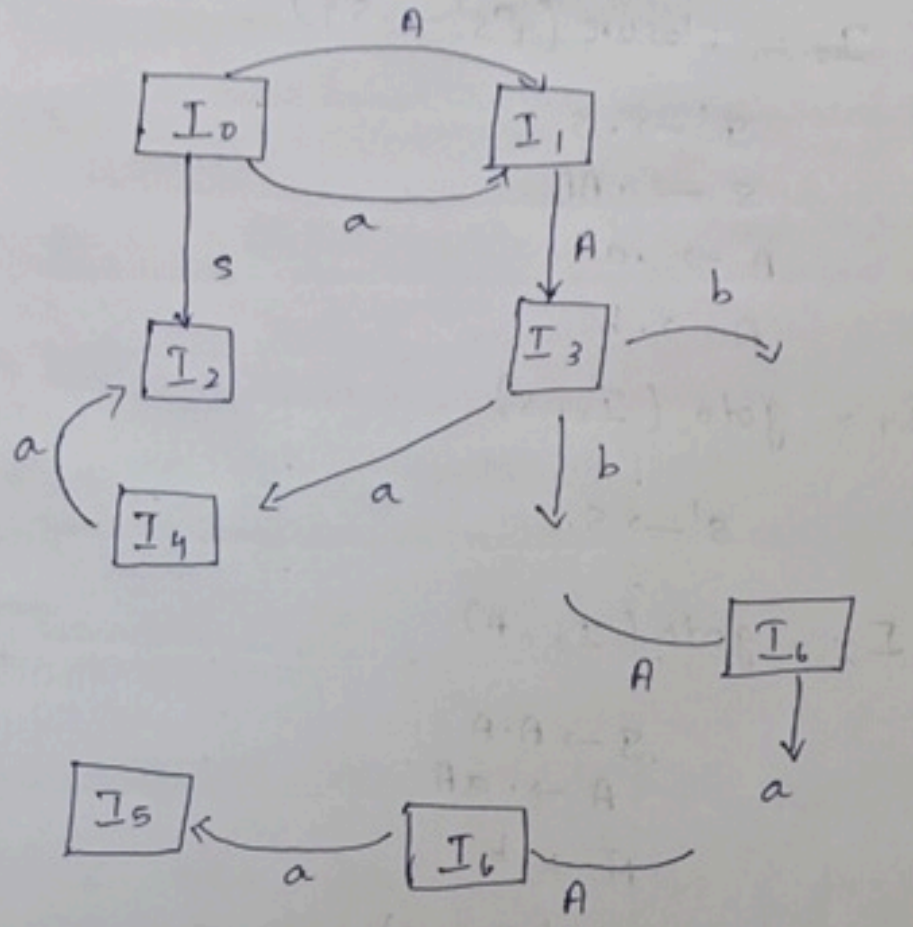
S → AA.

I_6 = goto (I_3, A)

A → aA.

I_3 has a self-loop on input a, because shifting a again produces the same item set.

2. LR(0) state transition diagram



The complete transitions, which define the canonical collection unambiguously

Current state	Symbol	Next state
I_0	S	I_1
I_0	A	I_2
I_0	a	I_3
I_0	b	I_4
I_2	A	I_5
I_2	a	I_3
I_2	b	I_4
I_3	A	I_6
I_3	a	I_3

3. LR(0) parsing table

Production numbers:

- r1 : S → AA
- r2 : A → aA
- r3 : A → b

In the table, S means shift r means reduce, and acc means accept.

State	ACTION a	ACTION b	ACTION $	GOTO S	GOTO A
0	S3	S4	–	1	2
1	–	–	acc	–	–
2	S3	S4	–	–	5
3	S3	S4	–	–	6
4	r3	r3	r3	–	–
5	r1	r1	r1	–	–
6	r2	r2	r2	–	–

There are no shift - reduce or reduce - reduce conflicts. Therefore, the grammar is LR(0).

4. Parse the input abab using LR(0)

Input : abab$

The stack contains states and grammar symbols.

- Shift: Consume the next input symbol.
- Reduce: replace the right - hand side of a production with its left - hand side.
- Accept: the input is successfully parsed.

Stack	Input	Action.
0	abab$	shift a, state 3
0a3	bab$	shift b, state 4
0a3b4	ab$	Reduce A->b
0a3A6	ab$	Reduce A->aA
0A2	ab$	shift a, state 3
0A2a3	b$	shift b, state 4
0A2a3b4	$	Reduce A->b
0A2a3A6	$	Reduce A->aA
0A2A5	$	Reduce S->AA
0S1	$	Accept

Result: abab is accepted by LR(0) parser.

The reduction correspond to:

$A \Rightarrow aA \Rightarrow ab$

$A \Rightarrow aA \Rightarrow ab$

$S \Rightarrow AA \Rightarrow abab.$

Part II : LR(1) Parsing

1. Construct the LR(1) items.

An LR(1) item has a production, a dot, and one lookahead symbol. We write it as $[X \to \alpha . \beta, a]$. For this grammar, the lookaheads determine when reduction are permitted.

Canonical collection of LR(1) items

I_0 = closure ($[S' \to . S, \$]$)

$[S' \to . S, \$]$

$[S \to . AA, \$]$

Since, FIRST $(A\$) = \{a, b\}$, the closure also contains:

$[A \to . a, a/b]$

$[A \to . b, a/b]$

I_1 = goto (I_0, S)

$[S' \to S ., \$]$

I_2 = goto (I_0, A)

$[S \to A . A\$]$

$[A \to . aA, \$]$

$[A \to . b, \$]$

I_3 = goto (I_0, a)

$[A \to a . A, a/b]$

$[A \to . aA, a/b]$

$[A \to . b, a/b]$

I_4 = goto $[I_0, b]$

$[A \rightarrow b\cdot, a/b]$

$I_5 = goto(I_2, A)$

$[S \rightarrow AA\cdot, \$]$

$I_6 = goto(I_2, a]$

$[A \rightarrow a\cdot A, \$]$

$[A \rightarrow \cdot aA, \$]$

$[A \rightarrow \cdot b\$]$

$I_7 = goto(I_2, b)$

$[A_2 \rightarrow b\cdot, \$]$

$I_8 = goto(I_3, A]$

$[A \rightarrow aA\cdot, a/b]$

$I_9 = goto(I_6, A)$

$[A \rightarrow aA\cdot, \$]$

Notice the key distinction btwn I_4 & I_7, I_8 & I_9: they have the same production in pairs but different lookaheads. LR(1) keeps these states seperate.

2. LR(1) state transition diagram

From state	Symbol	To state
0	S	1
0	A	2
0	a	3
0	b	4
2	A	5

2	a	6
2	b	7
3	A	8
3	a	3
3	b	4
6	A	9
6	a	6
6	b	7

This gives 10 states, I_0 through I_9. The self-loops on a occur in states I_3 & I_6.

3. LR(1) Parsing Table.

- $r1 : S \to AA$
- $r2 : A \to aA$
- $r3 : A \to b$

State	Action a	Action b	Action $	GOTO S	GOTO A
0	s3	s4	–	1	2
1	–	–	acc	–	–
2	s6	s7	–	–	5
3	s3	s4	–	–	8
4	r3	r3	–	–	–
5	–	–	r1	–	–
6	s6	s7	–	–	9
7	–	–	r3	–	–
8	r2	r2	–	–	–
9	–	–	r2	–	–

There are no parsing conflicts. Thus, the grammar is canonical LR(1) as well.

The important difference from LR(0) is that LR(1) reduces only on the specified look ahead symbol. For example, state 7 reduces A → b only on $, whereas state 4 reduces on a or b.

4. Parse the output abab using LR(1)

Input: abab$

Stack	Input	Action
0	abab$	shift a, state 2
0a3	bab$	shift b, state 4
0a3b4	ab$	Reduce A → b
0a3A8	ab$	Reduce A → aA
0A2	ab$	shift a, state 6
0A2a6	b$	shift b, state 7
0A2a6b7	$	reduce A → b
0A2a6A9	$	Reduce A → aA
0A2A5	$	Reduce S → AA
0S1	$	Accept.

Result: abab is accepted by the LR(1) parser.

Conclusion: Grammar S → AA, A → aA | b is both LR(0) and LR(1). Both successfully accept the input string abab.